LAB MANUAL: Counting Semaphores with Example

A **counting semaphore** is a synchronization object that controls access to a **limited number of resources** (like printers, connections etc.).

- It uses an **integer counter** to keep track of available resources.
- Threads **decrease the counter** when they access a resource.
- Threads **increase the counter** when they release it.
- If the counter is **zero**, threads will **wait** until it's positive.

Syntax to Use Counting Semaphore:

Declaration

```
sem_t sem;
```

Initialization

```
sem_init(&sem, pshared, value);
```

Arguments:

- `&sem`: Address of the semaphore variable.
- `pshared`:
  - `0` → semaphore is shared **between threads** in the same process.
  - `1` → shared between processes (not needed here).

- `value`: Initial count (e.g., 3 if we have 3 resources available).

Wait (Acquire)

```
sem_wait(&sem);
```

- Decreases the count by 1.
- If the value is 0, the thread **waits** (blocks) until another thread increases it.

Post (Release)

```
sem_post(&sem);
```

- Increases the count by 1.
- Signals that a resource has been released.

Destroy

```
sem_destroy(&sem);
```

Example: Printing Room with 3 Printers

Scenario

- There are **3 printers** in an office.
- **5 employees (threads)** want to print their documents.
- Only **3 can print at the same time**, others must wait.

Code: Counting Semaphore

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define EMPLOYEE_COUNT 5

sem_t printer_sem;  // Counting semaphore representing printers

void* print_document(void* arg) {
    int emp_id = *(int*)arg;

    // Request to access a printer
    printf("Employee %d: Waiting to print document...\n", emp_id);
    sem_wait(&printer_sem);

    printf("Employee %d: Using a printer.\n", emp_id);
    sleep(2);  // Simulate printing time

    printf("Employee %d: Done printing. Releasing printer.\n", emp_id);
    sem_post(&printer_sem);

    return NULL;
}

int main() {
    pthread_t employees[EMPLOYEE_COUNT];
    int ids[EMPLOYEE_COUNT];

    // Initialize semaphore with 3 printers
    sem_init(&printer_sem, 0, 3);

    // Create employee threads
    for (int i = 0; i < EMPLOYEE_COUNT; i++) {
        ids[i] = i + 1;
        pthread_create(&employees[i], NULL, print_document, &ids[i]);
    }
```

```
    // Wait for all employees to finish
    for (int i = 0; i < EMPLOYEE_COUNT; i++) {
        pthread_join(employees[i], NULL);
    }

    sem_destroy(&printer_sem);
    return 0;
}
```

Sample Output

```
Employee 1: Waiting to print document...
Employee 1: Using a printer.
Employee 2: Waiting to print document...
Employee 2: Using a printer.
Employee 3: Waiting to print document...
Employee 3: Using a printer.
Employee 4: Waiting to print document...
Employee 5: Waiting to print document...
Employee 1: Done printing. Releasing printer.
Employee 4: Using a printer.
Employee 2: Done printing. Releasing printer.
Employee 5: Using a printer.
Employee 3: Done printing. Releasing printer.
Employee 4: Done printing. Releasing printer.
Employee 5: Done printing. Releasing printer.
```

Dry Run Table (Printer Room Example)

Time	Employee	Printers Available	Action	Output
0s	E1	3 → 2	Access printer	Using a printer
0s	E2	2 → 1	Access printer	Using a printer
0s	E3	1 → 0	Access printer	Using a printer
0s	E4, E5	0	Waiting (blocked)	(Waiting...)
2s	E1	0 → 1	Releases printer	Done printing
2s	E4	1 → 0	Acquires printer	Using a printer
4s	E2	0 → 1	Releases printer	Done printing
4s	E5	1 → 0	Acquires printer	Using a printer
6s	E3	0 → 1	Releases printer	Done printing
8s	E4	1 → 2	Releases printer	Done printing
10s	E5	2 → 3	Releases printer	Done printing